Claude Guides — Claude Code (Complete)

claude-guides.com — all pages

kokoye2007

2026-06-17

Claude Guides — AI Cheatsheets, Commands & Hands-On Guides for Claude Code

> Source: https://claude-guides.com/ Crawled: 2026-06-17

Navigation

- Features
- Blog
- Contact

Claude Guides: AI Cheatsheets & Hands-On Guides

"Claude and AI guides plus practical cheatsheets — every Claude Code command, shortcut, slash command, MCP server, memory setting, skill, and agent, alongside deeper guides on prompt engineering, agents, and AI automation."

Available Languages: EN, ES, RU, 中文, 日本語

⏰ Claude Limits Status

- **Current Period:** —
- **Time Until Change:** —
- **System Status:** Checking...
- **Peak Hours:** 16:00–01:00 UTC
- **Best Time:** 01:00–16:00 UTC

⌨ Keyboard Shortcuts

Core

Shortcut	Action
Ctrl+C	Cancel input / generation
Ctrl+D	Exit session
Ctrl+L	Clear screen
Ctrl+O	Toggle verbose output
Ctrl+R	Reverse history search
Ctrl+G	Open prompt in editor
Ctrl+B	Background task
Ctrl+T	Toggle task list
Ctrl+V	Paste image
Ctrl+F ×2	Finish background agents
Esc	Undo / revert

Mode Switching

Shortcut	Action
Shift+Tab	Cycle permission modes
Alt+P	Switch model
Alt+T	Toggle thinking

Input

Shortcut	Action
Alt+Enter	New line (quick)
Ctrl+J	New line (control seq)

Prefixes

Prefix	Action
/	Slash command
!	Direct bash command
@	File mention + autocomplete

⬡ MCP Servers

Transports

Command	Description
`--transport http`	Remote HTTP (recommended)
`--transport stdio`	Local process
`--transport sse`	Remote SSE

Scopes

Location	Purpose
`.claude.json`	Project
`.mcp.json`	Shared / VCS
`~/.claude.json`	Global

Management

Command	Action
`/mcp`	Interactive UI
`claude mcp list`	List all servers
`claude mcp serve`	Run CC as MCP server

⚡ Slash Commands

Session

Command	Function
`/clear`	Clear conversation
`/compact [focus]`	Compress context
`/resume`	Resume / switch session
`/rename [name]`	Name current session
`/branch [name]`	Fork conversation (alias /fork)
`/cost`	Token usage stats

Command	Function
`/context`	Context usage visualizer
`/diff`	Interactive diff viewer
`/copy`	Copy last response
`/export`	Export conversation
`/rewind`	Checkpoint / rewind to previous point
`/status`	Show version, model, account status
`/stats`	Daily usage, streaks, preferences
`/tasks`	List background tasks

Settings

Command	Function
`/config`	Open settings
`/model [model]`	Switch model (←→ effort)
`/fast [on|off]`	Toggle fast mode
`/vim`	Toggle vim mode
`/theme`	Change color theme
`/permissions`	View / update permissions
`/effort [level]`	Set effort (low / med / high)
`/color [color]`	Prompt line color
`/keybindings`	Customize keybindings
`/terminal-setup`	Configure terminal keys
`/ide`	Manage IDE integrations
`/privacy-settings`	View / update privacy settings
`/remote-env`	Configure web session environment
`/desktop`	Continue in Desktop app

Tools

Command	Function
`/init`	Create CLAUDE.md
`/memory`	Edit CLAUDE.md
`/mcp`	Manage MCP servers

Command	Function
`/hooks`	Manage hooks
`/skills`	List available skills
`/agents`	Manage agents
`/add-dir <path>`	Add working directory
`/reload-plugins`	Hot-reload plugins
`/plugin`	Manage Claude Code plugins
`/install-github-app`	Set up GitHub Actions app
`/install-slack-app`	Install Slack app
`/web-setup`	Connect GitHub to web

Special

Command	Function
`/btw <question>`	Side question (no context cost)
`/plan [desc]`	Plan mode (+ auto-start)
`/ultraplan <prompt>`	Draft plan in browser, execute remotely
`/powerup`	Interactive lessons with animated demos
`/teleport`	Pull web session to terminal
`/autofix-pr [prompt]`	Auto-fix PR on CI failures
`/loop [interval]`	Schedule recurring task
`/voice`	Voice input (20 languages)
`/doctor`	Diagnose installation
`/pr-comments [PR]`	GitHub PR comments
`/remote-control`	Bridge to claude.ai/code
`/usage`	Plan limits & status
`/schedule`	Cloud scheduled tasks
`/security-review`	Security analysis
`/insights`	Analyze sessions & patterns
`/sandbox`	Toggle sandbox mode
`/extra-usage`	Configure extra usage quota
`/help`	Help + commands
`/feedback`	Send feedback (alias: /bug)

◎ Memory & Files

CLAUDE.md Locations

Location	Scope
`./CLAUDE.md`	Project (team-shared)
`~/.claude/CLAUDE.md`	Personal (all projects)
`/etc/claude-code/`	Managed (org-wide)

Rules & Imports

Item	Purpose
`.claude/rules/*.md`	Project rules
`~/.claude/rules/*.md`	User rules
`paths: frontmatter`	Path-specific rules
`@path/to/file`	Import in CLAUDE.md

Auto Memory

- `~/.claude/projects/<proj>/memory/MEMORY.md` + topic files, auto-loaded

◇ Workflows & Tips

Thinking & Effort

Action	Result
Alt+T	Toggle thinking
“ultrathink”	Max effort for one turn
Ctrl+O	View thinking (verbose)
`/effort`	low / med / high

Plan Mode

Command	Effect
Shift+Tab	Normal → Auto-accept → Plan
`--permission-mode plan`	Launch in plan mode

Git Worktrees

Option	Purpose
`--worktree name`	Isolated branch for feature
`isolation: worktree`	Agent in own worktree
`sparsePathsCheckout`	Only needed folders
`/batch`	Auto-create worktrees

Context Management

Command	Function
`/context`	Usage + optimization tips
`/compact [focus]`	Compress with focus
Auto-compact	~95% capacity
1M context	Opus 4.6 (Max / Team / Ent)
CLAUDE.md	Survives compaction

Session Tips

Command	Effect
`claude -c`	Continue last conversation
`claude -r "name"`	Resume by name
`/btw question`	Side question, no context cost

SDK / Headless

Command	Effect
`claude -p "prompt"`	Non-interactive mode
`--output-format json`	Structured output
`--max-budget-usd 5`	Cost limit
`cat file | claude -p`	Pipe input

Voice Mode

Feature	Details
`/voice`	Enable push-to-talk
Space (hold)	Record; release = send

Feature	Details
20 languages	EN, ES, FR, DE, CZ, PL...

⚙ Config & Environment

Config Files

File	Purpose
`~/.claude/settings.json`	User settings
`.claude/settings.json`	Project (shared)
`.claude/settings.local.json`	Local only
`~/.claude.json`	OAuth, MCP, state
`.mcp.json`	Project MCP servers

Key Settings

Setting	Purpose
`modelOverrides`	Map model selection → ID
`autoMemoryDirectory`	Custom memory directory
`worktree.sparsePaths`	Sparse checkout folders

Environment Variables

Variable	Purpose
`ANTHROPIC_API_KEY`	Authentication
`ANTHROPIC_MODEL`	Model selection
`CLAUDE_CODE_EFFORT_LEVEL`	low / med / high
`MAX_THINKING_TOKENS`	0 = off
`CLAUDE_CODE_MAX_OUTPUT_TOKENS`	default 32K
`CLAUDE_CODE_DISABLE_CRON`	Disable scheduled tasks

✦ Skills & Agents

Built-in Skills

Command	Function
`/simplify`	Code review (3 parallel agents)
`/batch`	Bulk changes (5–30 worktrees)
`/debug [desc]`	Debug from log
`/loop [interval]`	Recurring scheduled task
`/claude-api`	Load API + SDK reference

Custom Skill Locations

Location	Scope
`.claude/skills/<name>/`	Project skills
`~/.claude/skills/<name>/`	Personal skills

Skill Frontmatter

Property	Purpose
`description`	Auto-invoke trigger
`allowed-tools`	Skip permission prompts
`model`	Override model for skill
`effort`	Override effort level
`context: fork`	Run in subagent
`$ARGUMENTS`	User input placeholder
`${CLAUDE_SKILL_DIR}`	Skill directory
``!`cmd` ``	Dynamic context injection

Built-in Agents

Agent	Characteristics
Explore	Fast read-only (Haiku)
Plan	Research for plan mode
General	All tools, complex tasks
Bash	Terminal, separate context

Agent Frontmatter

Property	Values
`permissionMode`	default/acceptEdits/plan/dontAsk/bypass
`isolation: worktree`	Run in git worktree
`memory: user|project`	Persistent memory
`background: true`	Background task
`maxTurns`	Limit agent turns
`SendMessage`	Resume stopped agents

▸ CLI & Flags

Core Commands

Command	Function
`claude`	Interactive mode
`claude "prompt"`	With prompt
`claude -p "prompt"`	Headless
`claude -c`	Continue last
`claude -r "name"`	Resume by name
`claude update`	Update

Key Flags

Flag	Purpose
`--model`	Set model
`-w`	Git worktree
`-n / --name`	Session name
`--add-dir`	Add directory
`--agent`	Use an agent
`--allowedTools`	Pre-approve tools
`--output-format`	json / stream
`--json-schema`	Structured output
`--max-turns`	Limit turns

Flag	Purpose
`--max-budget-usd`	Cost limit
`--console`	Auth via Anthropic Console
`--verbose`	Verbose output
`--bare`	Minimal headless (no hooks/LSP)
`--channels`	Permission relay / MCP push
`--remote`	Web session
`--effort`	low / med / high / max
`--permission-mode`	plan / default / ...
`--dangerously-skip-permissions`	Skip all prompts ⚠

Quick Reference

Permission Modes: `default` prompts · `acceptEdits` auto-accept edits · `plan` read-only · `dontAsk` deny if not allowed · `bypassPermissions` skip all · CLI: `--dangerously-skip-permissions`

Environment Variables: - `ANTHROPIC_API_KEY` - `ANTHROPIC_MODEL` - `CLAUDE_CODE_EFFORT_LEVEL` (low/med/high) - `MAX_THINKING_TOKENS` (0=off) - `CLAUDE_CODE_MAX_OUTPUT_TOKENS` (default 32K) - `CLAUDE_CODE_DISABLE_CRON`

Credits: Paul Larionov · Silikonoviy Meshok

Claude Code Cheatsheet — Features & Guide

> Source: https://claude-guides.com/cheatsheet-landing-page.html Crawled: 2026-06-17

Navigation

- Features
- Blog
- Contact

Claude Code Cheatsheet

Master Claude Code with comprehensive guides covering keyboard shortcuts, commands, MCP servers, memory management, workflows, and best practices.

⌨️ Keyboard Shortcuts

Quick Reference

Master essential keyboard shortcuts for Mac and Windows. Speed up your workflow with core navigation, mode switching, and input shortcuts.

- Core shortcuts (Cancel, Exit, Clear)
- Mode switching (Permission, Model, Thinking)
- Input methods & prefixes
- Navigation & search commands

View All Shortcuts

/ Slash Commands

Power Tools

Execute powerful operations with slash commands. Manage tasks, files, configuration, and more directly from your terminal.

- Task management (/clear, /todo)

- File operations (/ls, /cat, /git)
- Configuration & tools (/settings, /help)
- AI operations (/ask, /improve, /test)

Explore Commands

🔗 MCP Servers

Integrations

Connect with external services and tools. MCP servers extend Claude Code functionality with GitHub, databases, and more integrations.

- GitHub integration & workflows
- Database & file system access
- Web APIs & external services
- Custom MCP server setup

Learn Integration

💾 Memory Management

Optimization

Understand how Claude Code manages context and memory. Learn to optimize for longer sessions and complex projects.

- Context window management
- Memory persistence & projects
- Token usage optimization
- Session persistence

Read Full Guide

⚙️ Configuration & Settings

Customization

Customize Claude Code to match your workflow. Configure settings, hooks, preferences, and environment variables.

- Settings.json configuration
- Shell hooks & automation
- Environment variables setup

- IDE integrations & extensions

Setup Guide

🚀 Advanced Workflows

Best Practices

Discover advanced patterns and workflows. Learn expert strategies for debugging, testing, code review, and complex projects.

- Debugging techniques & tools
- Testing & CI/CD integration
- Code review workflows
- Multi-file project management

Explore Workflows

🎯 Prompt Engineering

Skills

Master the art of crafting effective prompts. Learn techniques for better results with Claude AI across different scenarios.

- Prompt structure & clarity
- Context & examples
- Role-based prompting
- Iterative refinement

Learn Techniques

🔍 Hidden Features

Pro Tips

Uncover lesser-known features and power-user tips. Level up your Claude Code mastery with advanced techniques.

- Undocumented features
- Keyboard modifier combinations
- Command chaining techniques
- Performance secrets

Discover Tips

Cheatsheet Topics Overview

Topic	Coverage	Difficulty	Time to Master
Keyboard Shortcuts	Mac & Windows shortcuts, prefixes	Beginner	15 min
Slash Commands	30+ commands for all operations	Beginner	30 min
MCP Servers	Integration setup & usage	Intermediate	1 hour
Memory Management	Context, projects, persistence	Intermediate	45 min
Configuration	Settings, hooks, environment	Intermediate	1 hour
Advanced Workflows	Debugging, testing, CI/CD	Advanced	2+ hours
Prompt Engineering	Techniques & best practices	Advanced	2+ hours
Hidden Features	Pro tips & undocumented features	Advanced	1+ hour

Ready to Master Claude Code?

Start with the main cheatsheet page to explore all keyboard shortcuts, commands, and configurations in one comprehensive reference.

Open Main Cheatsheet

Claude Code Cheatsheet — A comprehensive reference for Claude Code users

Made with ❤️ by Paul Larionov | GitHub

Best Practices for Using Claude Code — Complete Guide

> Source: https://claude-guides.com/blog/best-practices.html Crawled: 2026-06-17

Master the Claude Code CLI with expert strategies to boost productivity, streamline workflows, and leverage advanced features effectively.

1. Master Keyboard Shortcuts for Speed

Learning keyboard shortcuts is the fastest way to boost your productivity in Claude Code. Instead of typing full commands, use these essential shortcuts:

- `Ctrl+C` (Mac: `^C`) — Cancel input or generation instantly
- `Ctrl+G` (Mac: `^G`) — Open your prompt in an external editor for complex queries
- `Ctrl+L` (Mac: `^L`) — Clear the screen to reduce visual clutter
- `Alt+P` (Mac: `⌥P`) — Switch between AI models without restarting
- `Ctrl+F ×2` (Mac: `^F ×2`) — Finish all background agents at once

💡 **Pro Tip:** "Memorize the shortcuts you use most frequently. Even 3-4 shortcuts can dramatically reduce typing and improve workflow speed."

2. Use Plan Mode for Complex Projects

When tackling multi-step tasks or architectural decisions, use plan mode to get a structured approach before diving into implementation:

- Launch with `/launch-plan` to get a detailed implementation strategy
- Plan mode is read-only, so you can review the approach without committing to changes
- Perfect for understanding project structure before making large refactors
- Great for breaking down ambiguous requirements into clear action items

3. Leverage Agents for Parallel Work

Instead of handling everything sequentially, delegate to agents to work in parallel:

- Use `/general` agent for complex multi-tool tasks
- Use `/explore` agent for fast codebase exploration (Haiku-based, quick read-only)
- Use `/plan` agent for research in plan mode without committing changes
- Set maximum turns with `--max-agent-turns` to keep agents focused
- Run in isolated worktrees with `--agent-worktree` for safe experimentation

💡 **Pro Tip:** "Use agents for parallel tasks like searching documentation while exploring code. This saves time by avoiding sequential bottlenecks."

4. Optimize Memory Management

Context window is precious. Use these strategies to keep your context window clean and efficient:

- `/remember` — Save important context that persists across sessions
- `/memory` — View and manage your persistent memory
- Use `CLAUDE_CODE_EFFORT_LEVEL=med` for balanced thinking depth vs. speed
- Auto-compact at ~95% capacity to prevent context overflow
- Use `MAX_THINKING_TOKENS=0` to disable extended thinking if not needed
- Archive old conversations to keep working context fresh

5. Master Slash Commands for Workflow Control

Slash commands are shortcuts to common workflows. Key ones to master:

- `/review` — Request code review with multiple agents in parallel
- `/debug` — Debug issues directly from error logs
- `/commit` — Create a git commit with your changes
- `/help` — Get help on Claude Code features
- `/fast` — Toggle fast mode for quicker responses
- `/memory` — Manage persistent memory across sessions

6. Use Permission Modes Strategically

Different permission modes suit different workflows. Choose wisely:

- `default` — Prompts for each action (safe, good for learning)

- `acceptEdits` — Auto-accept file edits without prompting
- `plan` — Read-only mode, perfect for exploration
- `dontAsk` — Denies actions if not pre-approved
- `bypassPermissions` — Maximum autonomy (use with caution)

⚠️ **Security Note:** "Use `bypassPermissions` only in trusted, isolated environments. For production code, prefer `default` or `acceptEdits`."

7. Configure Environment Variables for Your Workflow

Customize Claude Code behavior with environment variables:

- `ANTHROPIC_API_KEY` — Set your API key securely
- `ANTHROPIC_MODEL` — Default model (e.g., `claude-opus-4-6`)
- `CLAUDE_CODE_EFFORT_LEVEL` — Thinking effort (low/med/high)
- `MAX_THINKING_TOKENS` — Extended thinking budget (0=off, 1M=max)
- `CLAUDE_CODE_MAX_OUTPUT_TOKENS` — Response length (default 32K)

8. Use MCP Servers for Extended Capabilities

MCP (Model Context Protocol) servers extend Claude Code with custom tools:

- Configure MCP servers in your project or user settings
- Popular MCPs: filesystem access, database tools, API integrations
- Use `@` mentions to reference files and data efficiently
- Combine multiple MCPs for complex workflows (e.g., git + database)

9. Effective Prompt Engineering for Claude Code

Better prompts lead to better results. Use these techniques:

- **Be specific:** "Add pagination to the user list component" beats "improve the component"
- **Provide context:** Include relevant file paths, error messages, or requirements
- **Use multi-line mode:** Press `Alt+Enter` (Mac: `\Enter`) to write longer prompts
- **Reference files:** Use `@filename` to include file context automatically
- **Ask for the format:** Specify JSON, markdown, or code format for structured responses

10. Git Integration and Version Control

Claude Code integrates seamlessly with git for safe development:

- Use `/commit` to create semantic commits with clear messages
- Claude Code respects your `.gitignore` — sensitive files are protected
- Use worktrees (`--agent-worktree`) to sandbox experimental changes
- Create feature branches before starting major work
- Use git hooks to enforce code quality standards

11. Performance Optimization Tips

Keep Claude Code running smoothly:

- Close unused windows and reduce background processes
- Use `--bare` flag for minimal headless mode (no hooks/LSP)
- Limit agent turns with `--max-agent-turns 3` to avoid endless loops
- Use the `/explore` agent instead of `/general` for faster codebase searches
- Cache frequently accessed data in persistent memory with `/remember`

12. Continuous Learning and Community

Stay updated with Claude Code features and best practices:

- Check the Claude Code Cheatsheet regularly for new commands
- Follow Claude Code updates on Paul Larionov's X (Twitter)
- Explore Claude Certified Architect for advanced patterns
- Experiment with different agents and permission modes in your workflows
- Share your workflows and tips with the community

Common Pitfalls to Avoid

- **Not using keyboard shortcuts:** Typing full commands wastes time every single day
- **Ignoring plan mode:** For complex tasks, planning saves debugging time later
- **Overloading context:** Keep conversations focused; archive old ones
- **Not reading error messages:** Claude Code provides clear error context — use it
- **Using manual approvals for every change:** Once you're comfortable, use `acceptEdits` mode

- **Forgetting to commit:** Make frequent, small commits instead of massive ones

Conclusion

“Claude Code is a powerful tool that rewards mastery. Start with keyboard shortcuts and plan mode, then gradually explore agents, memory management, and advanced configurations.”

Remember: **the best practices are the ones you actually use consistently**. Pick 2-3 techniques that resonate with your workflow, master them, then gradually add more to your toolkit.

Hidden and Underused Features in Claude Code — Advanced Guide

> Source: https://claude-guides.com/blog/hidden-features.html Crawled: 2026-06-17

Overview

According to the article by Boris Cherny from Anthropic, “Most people use Claude Code as a simple coding assistant” but it contains numerous powerful capabilities for those who leverage them strategically.

Key Features Highlighted

The guide covers 15 advanced features:

1. **Mobile App Coding** — Write and modify code from iOS or Android
2. **Cross-Device Sessions** — Use `claude --teleport` or `/teleport` to move between devices
3. **Automation Commands** — `/loop` and `/schedule` enable repeated task execution
4. **Hooks** — Inject logic into agent lifecycle for observability and control
5. **Cowork Dispatch** — Remote execution from phones or browsers
6. **Chrome Extension** — Direct browser integration for frontend debugging
7. **Auto Web Server Testing** — Automated server startup and testing
8. **Session Forking** — Branch work using `/branch` or `claude --resume --fork-session`
9. **Side Queries** — Use `/btw` for quick clarifications without interrupting main tasks
10. **Git Worktrees** — Support for parallel development environments
11. **Batch Processing** — `/batch` distributes work across multiple agents
12. **Bare Mode** — `--bare` flag accelerates execution
13. **Multi-Repo Context** — `--add-dir` provides cross-repository access
14. **Custom Agents** — Define specialized agents via `.claude/agents`
15. **Voice Input** — Code by speaking using `/voice` or desktop/iOS features

Central Insight

The article emphasizes treating Claude Code as “an autonomous system rather than a chatbot,” with automation and parallel execution distinguishing power users from casual users.

Claude Code Architectural Patterns & Advanced Workflows — Complete Guide

> Source: https://claude-guides.com/blog/claude-code-architectural-patterns.html
> Crawled: 2026-06-17

Master the convergent Research → Plan → Execute pattern, autonomous agents, subagents, and production-ready AI development strategies that power enterprise workflows.

1. Core Architectural Pattern: Research → Plan → Execute → Review → Ship

The convergent pattern across all major Claude Code workflows is a five-stage process that ensures systematic, high-quality outcomes:

1. **Research:** Gather information, explore codebase, understand requirements
2. **Plan:** Design architecture, identify critical files, outline implementation strategy
3. **Execute:** Implement changes, write code, update files systematically
4. **Review:** Check quality, test functionality, verify correctness
5. **Ship:** Commit, push, and deploy to production

💡 **Pro Tip:** This pattern scales from single-file edits to enterprise multi-agent deployments. Use it consistently for predictable, reliable outcomes.

2. Autonomous Agents & Subagents

Agents are the foundation of autonomous AI workflows in Claude Code. A subagent is:

> "Autonomous actor in fresh isolated context — custom tools, permissions, model, memory, and persistent identity"

Key Characteristics:

- **Fresh Context:** Each subagent starts with a clean slate, no history bloat

- **Custom Tools:** Configure exactly which tools (filesystem, git, API access) each agent can use
- **Isolated Permissions:** Fine-grained control over what each agent can access
- **Persistent Identity:** Agents maintain identity across tasks, enabling multi-step workflows
- **Shared Coordination:** Multiple agents work in parallel on the same codebase with shared task tracking

Agent Teams for Parallel Development:

Launch multiple agents simultaneously, each with its own worktree (isolated git branch). This enables true parallelization without conflicts:

```
# Agent A works on feature/auth
# Agent B works on feature/database
# Agent C works on feature/api
# All merge back to main when ready
```

💡 **Pro Tip:** Use agent teams for features that can be developed independently. Each agent gets a dedicated git worktree, eliminating merge conflicts during development.

3. Auto Mode: Safety Without Prompts

By default, Claude Code asks for permission before executing sensitive operations. **Auto Mode** replaces manual permission prompts with a background safety classifier:

```
claude --enable-auto-mode
```

This allows agents to work autonomously while maintaining safety guardrails. The classifier evaluates risk and auto-approves safe operations, blocking suspicious ones.

4. Organization & Context Management

Commands vs. Skills vs. Memory

Claude Code has three ways to inject knowledge into existing context:

Commands

“Knowledge injected into existing context — simple user-invoked prompt templates for workflow orchestration.”

Use when: You need a quick, repeatable workflow. Examples: `/batch`, `/simplify`, `/commit`

Skills

"Knowledge injected into existing context — configurable, preloadable, auto-discoverable, with context forking."

Use when: You need more complexity and customization than commands. Skills can be triggered automatically based on conditions.

Memory (CLAUDE.md Files & @path Imports)

Persistent context via CLAUDE.md files and @path imports with auto-memory and rules organization.

Use when: You need rules, conventions, or project-specific knowledge that applies across all sessions.

Memory Best Practices:

- Create `CLAUDE.md` in your project root with project conventions, style guides, and rules
- Use `@path imports` to inject context from specific files without bloating memory
- Organize memory hierarchically: project-level, team-level, personal preferences
- Keep memory under 5K tokens for optimal context efficiency

5. Execution & Scaling: From Local to Cloud

Scheduled Tasks: /loop vs. /schedule

- **/loop:** Runs prompts locally on your machine, up to 3 days of persistence. Perfect for polling, monitoring, and recurring tasks that don't require continuous uptime
- **/schedule:** Runs tasks in the cloud even when your machine is offline. For workflows that must run continuously regardless of your local session state

Example: Use `/loop 5m "check deployment status"` to poll status every 5 minutes for the next 3 days.

Parallel Development with Git Worktrees

Claude Code automatically creates isolated git worktrees for each agent, enabling truly parallel development:

```
# Each agent gets its own worktree
agent-1: .git/worktrees/feature-auth
agent-2: .git/worktrees/feature-database
agent-3: .git/worktrees/feature-api

# All changes merge cleanly back to main
```

Remote Control: Continue From Any Device

Resume sessions from any device without loss of context:

```
/remote-control
# or
/rc
```

Useful for picking up work on the go, switching between desktop/laptop, or handing off work to teammates.

6. Hot Features & Advanced Capabilities

Multi-Agent Code Review

Leverage Claude Code's code review feature to have multiple agents analyze PRs simultaneously, catching:

- Logic bugs and edge case failures
- Security vulnerabilities (SQL injection, XSS, etc.)
- Performance regressions and memory leaks
- Style violations and architectural drift

Ultraplan: Cloud-Based Planning

Draft implementation plans in the cloud with:

- Browser-based review interface
- Inline comments and suggestions
- Real-time collaboration with teammates
- Version history and approval workflows

Claude Code Web

Run tasks on cloud infrastructure with:

- Scheduled tasks that execute without your machine running
- Parallel sessions for independent workflows
- Higher resource limits than local execution
- Persistent logs and audit trails

Agent SDK (Python/TypeScript)

Build production AI agents using Claude Code as a library. Integrate autonomous agents directly into your applications with:

- Full control over agent behavior and permissions
- Custom tool definitions and integrations
- Embedded model selection and inference parameters
- Event streaming and real-time monitoring

Channels: Push Events Into Running Sessions

Integrate Telegram, Discord, and other chat platforms to push events directly into Claude Code sessions. Claude reacts autonomously to incoming messages, making your workflow context-aware and responsive.

Computer Use (Beta)

Let Claude control your screen on macOS for end-to-end automation of GUI-based workflows. Useful for testing, browser automation, and workflows that require visual feedback.

7. MCP Servers: Extend Claude Code Capabilities

MCP (Model Context Protocol) servers connect Claude Code to external tools, databases, and APIs. Configure them in `.mcp.json`:

```
{
  "mcpServers": {
    "github": { "command": "mcp-github" },
    "slack": { "command": "mcp-slack" },
    "postgres": { "command": "mcp-postgres" }
  }
}
```

Common integrations:

- Version control (GitHub, GitLab)
- Communication (Slack, Discord, Email)
- Databases (PostgreSQL, MongoDB, Firebase)
- Cloud platforms (AWS, GCP, Azure)
- Project management (Jira, Linear, Asana)
- APIs and webhooks

8. Configuration & Settings

Hierarchical Settings in .claude/settings.json

Configure Claude Code behavior globally or per-project:

- **Permissions:** Which operations require approval
- **Model Config:** Default model, temperature, max tokens
- **Output Styles:** Formatting, colors, verbosity
- **Keyboard Bindings:** Custom shortcuts for frequent actions
- **Auto-Behaviors:** Hooks that run before/after operations

9. Pro Tips for Production Excellence

Tip 1: Simplify Code Quality with /simplify

Review changed code for reuse, quality, and efficiency, then fix issues automatically. Use this after implementing features to ensure clean, maintainable code.

Tip 2: Batch Operations with /batch

Apply the same transformation across multiple files. Great for refactoring variable names, updating imports, or applying consistent formatting.

Tip 3: Checkpointing for Safety

Use `/rewind` or press `Esc Esc` to track and rewind file edits automatically. This gives you a safety net for exploratory changes without committing them.

Tip 4: Voice Dictation

Enable voice input with `/voice`. Supports 20+ languages and rebindable keys for hands-free control during complex problem-solving sessions.

Tip 5: Strategic Model Selection

Different models have different costs and capabilities:

- **Haiku:** Fast, lightweight work (queries, summaries, simple edits)
- **Sonnet:** General-purpose (coding, analysis, detailed writing)
- **Opus:** Deep reasoning required (complex architecture, novel problems)

Switch models deliberately at the start of sessions to optimize for cost vs. capability.

10. Workflow Intelligence Across Teams

Different teams emphasize distinct approaches to Claude Code workflows:

Everything Claude Code (148k references)

Focus: Instinct scoring, AgentShield safety, multi-language rules. Best for: Large teams with diverse tech stacks.

Superpowers (143k references)

Focus: TDD-first, Iron Laws, whole-plan review. Best for: Quality-obsessed teams, large refactors, greenfield projects.

Get Shit Done (50k references)

Focus: Fresh 200K contexts, wave execution, XML plans. Best for: Speed-first teams, MVPs, rapid iteration.

BMAD-METHOD (44k references)

Focus: Full SDLC, agent personas, 22+ platforms. Best for: Enterprise teams, compliance-heavy projects, complex workflows.

Conclusion: Building for Scale

Mastering Claude Code's architectural patterns enables you to:

- Build autonomous workflows that scale from single features to enterprise systems
- Leverage agents and subagents for true parallel development
- Integrate external systems and data sources seamlessly via MCP servers
- Maintain code quality and safety while accelerating development speed
- Adopt the Research → Plan → Execute → Review → Ship pattern for predictable outcomes

🚀 **Next Steps:** Start with a single agent on a small feature, master the Research → Plan → Execute pattern, then scale to multi-agent teams and cloud-based workflows. Each step is a natural progression that builds on the previous one.

Last updated: April 12, 2026